

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 10 – Nilai Max/Min



DWI SASMITA AGUS ANGGRAINI

109082500026

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pencarian adalah suatu proses yang lazim dilakukan di dalam kehidupan sehari-hari. Contoh penggunaannya dalam kehidupan nyata sangat beragam, misalnya pencarian file di dalam directory komputer, pencarian suatu teks di dalam sebuah dokumen, pencarian buku pada rak buku, dan masih banyak contoh lainnya.

Pada modul ini akan dipelajari salah satu algoritma pencarian nilai terkecil atau terbesar pada sekumpulan data, atau yang biasa disebut sebagai pencarian nilai ekstrim. Algoritma ini merupakan salah satu konsep dasar yang penting untuk dipahami dalam dunia pemrograman dan pengolahan data.

Ide dari algoritma ini sangat sederhana. Karena data harus diproses secara sekuensial, maka nilai atau indeks ke nilai maksimum dari data yang telah diproses disimpan untuk dibandingkan dengan data berikutnya. Nilai yang berhasil disimpan sampai algoritma tersebut berakhir adalah nilai maksimum yang dicari.

B. Guided

1. Guided 1

```
package main

import "fmt"

func nilaiTerbesar(array []int) (int, int) {
    var idx int
    var nilaiTerbesar int = array[0]
    var idxDitemukan int = 0
    for idx = 1; idx < len(array); idx++ {
        if array[idx] > nilaiTerbesar {
            nilaiTerbesar = array[idx]
            idxDitemukan = idx
        }
    }
    return nilaiTerbesar, idxDitemukan
}

func nilaiTerkecil(array []int) (int, int) {
    var idx int
    var nilaiTerkecil int = array[0]
    var idxDitemukan int = 0
    for idx = 1; idx < len(array); idx++ {
        if array[idx] < nilaiTerkecil {
            nilaiTerkecil = array[idx]
            idxDitemukan = idx
        }
    }
    return nilaiTerkecil, idxDitemukan
}

func main() {
    var arrAngka [5]int = [5]int{45, 23, 98, 66, 12}

    var nilaiMaks, idxNilaiMaks int = nilaiTerbesar(arrAngka[:])
    var nilaiMin, idxNilaiMin int = nilaiTerkecil(arrAngka[:])

    fmt.Printf("Nilai terkecil : %d ditemukan pada index ke-%d", nilaiMin,
idxNilaiMin)
    fmt.Println()
    fmt.Printf("Nilai terbesar : %d ditemukan pada index ke-%d", nilaiMaks,
idxNilaiMaks)
}
```

Output :

```
PS D:\ALPRO2\109082500026_Dwi-Sasmita-Agus-Anggraini\Week10-Modul10\Guided\Guided-1> go run guided1.go
Nilai terkecil : 12 ditemukan pada index ke-4
Nilai terbesar : 98 ditemukan pada index ke-2
PS D:\ALPRO2\109082500026_Dwi-Sasmita-Agus-Anggraini\Week10-Modul10\Guided\Guided-1> █
```

Penjelasan:

Program ini menggunakan dua fungsi, `nilaiTerbesar` dan `nilaiTerkecil`, yang masing-masing menerima slice integer dan mengembalikan dua nilai sekaligus yaitu nilai yang ditemukan beserta indeksinya. Kedua fungsi bekerja dengan cara yang sama: menetapkan elemen pertama array sebagai nilai acuan awal, lalu melakukan iterasi mulai indeks ke-1 untuk membandingkan setiap elemen, jika ditemukan nilai yang lebih besar (atau lebih kecil), nilai acuan dan indeks diperbarui. Di fungsi `main`, array berisi 5 angka dikonversi menjadi slice lalu diproses oleh kedua fungsi tersebut.

2. Guided 2

```
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim  string
    IPK  float64
}

type arrMahasiswa [3]mahasiswa

func IPKTerbesar(array []mahasiswa) (mahasiswa, int) {
    var mhsTerbesar mahasiswa = array[0]
    var idxDitemukan int = 0
    var i int

    for i = 1; i < len(array); i++ {
        if array[i].IPK > mhsTerbesar.IPK {
            mhsTerbesar = array[i]
            idxDitemukan = i
        }
    }

    return mhsTerbesar, idxDitemukan
}

func IPKTerkecil(array []mahasiswa) (mahasiswa, int) {
    var mhsTerkecil mahasiswa = array[0]
    var idxDitemukan int = 0
    var i int

    for i = 1; i < len(array); i++ {
        if array[i].IPK < mhsTerkecil.IPK {
            mhsTerkecil = array[i]
            idxDitemukan = i
        }
    }

    return mhsTerkecil, idxDitemukan
}

func main() {
    var arrMhs arrMahasiswa
    var i, j int
    for i = 0; i < len(arrMhs); i++ {
        fmt.Println("DATA MAHASISWA INDEKS KE-", i)
```

Output :

```
GetFileAttributesEx guided2.go: The system cannot find the file specified.
PS D:\ALPRO2\109082500026_Dwi-Sasmita-Agus-Anggraini\Week10-Modul10\Guided\Guided-2> go run guided2.go
DATA MAHASISWA INDEKS KE- 0
Masukkan nama : Dwisasmitaagusanggraini
Masukkan NIM :109082500026
Masukkan IPK : 4
=====
DATA MAHASISWA INDEKS KE- 1
Masukkan nama : nepy
Masukkan NIM :109082500152
Masukkan IPK : 2
=====
DATA MAHASISWA INDEKS KE- 2
Masukkan nama : 
```

Penjelasan:

Program ini menggunakan struct mahasiswa untuk menyimpan data nama, NIM, dan IPK, serta mendefinisikan tipe array arrMahasiswa berkapasitas 3 elemen. Dua fungsi IPKTerbesar dan IPKTerkecil bekerja dengan logika yang sama seperti program sebelumnya, namun kali ini perbandingan dilakukan pada field IPK dari setiap elemen struct, bukan nilai integer biasa. Di fungsi main, array mahasiswa diisi datanya menggunakan perulangan, lalu kedua fungsi dipanggil untuk mencari mahasiswa dengan IPK tertinggi dan terendah, dan hasilnya ditampilkan beserta indeksinya.

C. Unguided

1. Soal Unguided 1

- 1) Sebuah program digunakan untuk mendata berat anak kelinci yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat anak kelinci yang akan dijual.

Masukan terdiri dari sekumpulan bilangan, yang mana bilangan pertama adalah bilangan bulat N yang menyatakan banyaknya anak kelinci yang akan ditimbang beratnya. Selanjutnya N bilangan riil berikutnya adalah berat dari anak kelinci yang akan dijual.

Keluaran terdiri dari dua buah bilangan riil yang menyatakan berat kelinci terkecil dan terbesar.

Jawaban :

```
package main

import "fmt"

func main() {
    var N, i, j int
    var kelinci [1000]float64
    var min, max float64

    fmt.Print("input N: ")
    fmt.Scan(&N)

    for i = 0; i < N; i++ {
        fmt.Printf("input berat ke-%d: ", i+1)
        fmt.Scan(&kelinci[i])
    }

    min = kelinci[0]
    max = kelinci[0]

    for j = 1; j < N; j++ {
        if kelinci[j] < min {
            min = kelinci[j]
        }

        if kelinci[j] > max {
            max = kelinci[j]
        }
    }
}
```

```
    fmt.Println(min, max)
}
```

Output :

```
PS D:\ALPRO2\109082500026_Dwi-Sasmita-Agus-Anggraini\Week10-Modul10\Unguided\Unguided-1> go run Unguided1.go
input N: 5
input berat ke-1: 1
input berat ke-2: 2
input berat ke-3: 3
input berat ke-4: 4
input berat ke-5: 5
1 5
PS D:\ALPRO2\109082500026_Dwi-Sasmita-Agus-Anggraini\Week10-Modul10\Unguided\Unguided-1> █
```

Penjelasan :

Program ini meminta input jumlah kelinci N, lalu menyimpan berat masing-masing kelinci ke dalam array kelinci melalui perulangan pertama. Setelah semua data tersimpan, perulangan kedua membandingkan setiap elemen mulai indeks ke-1 dengan nilai min dan max yang awalnya diset dari elemen pertama, jika ditemukan nilai lebih kecil maka min diperbarui, jika lebih besar maka `max` diperbarui. Di akhir program, nilai berat terkecil dan terbesar ditampilkan sekaligus dalam satu baris.

2. Soal Unguided 2

- 2) Sebuah program digunakan untuk menentukan tarif ikan yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat ikan yang akan dijual.

Masukan terdiri dari dua baris, yang mana baris pertama terdiri dari dua bilangan bulat x dan y. Bilangan x menyatakan banyaknya ikan yang akan dijual, sedangkan y adalah banyaknya ikan yang akan dimasukkan ke dalam wadah. Baris kedua terdiri dari sejumlah x bilangan riil yang menyatakan banyaknya ikan yang akan dijual.

Keluaran terdiri dari dua baris. Baris pertama adalah kumpulan bilangan riil yang menyatakan total berat ikan di setiap wadah (jumlah wadah tergantung pada nilai x dan y, urutan ikan yang dimasukkan ke dalam wadah sesuai urutan pada masukan baris ke-2). Baris kedua adalah sebuah bilangan riil yang menyatakan berat rata-rata ikan di setiap wadah.

Jawaban :

```
package main

import (
    "fmt"
    "math"
)

func main() {
    var x, y, i, jmlWadah, wadahKe, ikanDiWadah int
    var rata2 float64
    var ikan [1000]float64
    var totalPerWadah [1000]float64

    fmt.Print("input x & y: ")
    fmt.Scan(&x, &y)

    for i = 0; i < x; i++ {
        fmt.Printf("input berat ke-%d: ", i+1)
        fmt.Scan(&ikan[i])
    }

    jmlWadah = int(math.Ceil(float64(x) / float64(y)))

    for i = 0; i < x; i++ {
        wadahKe = i / y
        totalPerWadah[wadahKe] += ikan[i]
    }
}
```

```

    for i = 0; i < jmlWadah; i++ {
        if i > 0 {
            fmt.Print(" ")
        }
        fmt.Print(totalPerWadah[i])
    }

    fmt.Println()

    for i = 0; i < jmlWadah; i++ {
        ikanDiWadah = y
        if i == jmlWadah-1 {
            ikanDiWadah = x - (i * y)
        }

        rata2 = totalPerWadah[i] / float64(ikanDiWadah)

        if i > 0 {
            fmt.Print(" ")
        }

        fmt.Print(rata2)
    }

    fmt.Println()
}

```

Output :

```

PS D:\ALPRO2\109082500026_Dwi-Sasmita-Agus-Anggraini\Week10-Modul10\Unguided\Unguided-2> go run Unguided2.go
input x & y: 1 2
input berat ke-1: 1.0
1
1
PS D:\ALPRO2\109082500026_Dwi-Sasmita-Agus-Anggraini\Week10-Modul10\Unguided\Unguided-2>

```

Penjelasan :

Program ini menerima dua input x (jumlah ikan) dan y (kapasitas per wadah), lalu menyimpan berat tiap ikan ke dalam array. Jumlah wadah dihitung agar sisa ikan yang tidak memenuhi satu wadah penuh tetap mendapat wadah tersendiri. Setiap ikan kemudian dikelompokkan ke wadahnya masing-masing dan beratnya diakumulasi ke array totalPerWadah. Di akhir, program menampilkan total berat per wadah, dan rata-rata berat per wadah dengan memperhitungkan bahwa wadah terakhir mungkin tidak penuh.

3. Soal Unguided 3

- 3) Pos Pelayanan Terpadu (posyandu) sebagai tempat pelayanan kesehatan perlu mencatat data berat balita (dalam kg). Petugas akan memasukkan data tersebut ke dalam array. Dari data yang diperoleh akan dicari berat balita terkecil, terbesar, dan reratanya.

Buatlah program dengan spesifikasi subprogram sebagai berikut:

```
type arrBalita [100]float64

func hitungMinMax(arrBerat arrBalita; bMin, bMax *float64) {
  /* I.S. Terdefinisi array dinamis arrBerat
     Proses: Menghitung berat minimum dan maksimum dalam array
     F.S. Menampilkan berat minimum dan maksimum balita */
  ...
}

function rerata (arrBerat arrBalita) real {
  /* menghitung dan mengembalikan rerata berat balita dalam array */
  ...
}
```

Perhatikan sesi interaksi pada contoh berikut ini (teks bergaris bawah adalah input/read)

```
Masukan banyak data berat balita : 4
Masukan berat balita ke-1: 5.3
Masukan berat balita ke-2: 6.2
Masukan berat balita ke-3: 4.1
Masukan berat balita ke-4: 9.9
Berat balita minimum: 4.10 kg
Berat balita maksimum: 9.90 kg
Rerata berat balita: 6.38 kg
```



Jawaban :

```
package main

import "fmt"

type arrBalita [100]float64

func hitungMinMax(arrBerat arrBalita, bMin, bMax *float64) {
    var n, i int

    n = int(arrBerat[0])
    *bMin = arrBerat[1]
    *bMax = arrBerat[1]

    for i = 1; i <= n; i++ {
        if arrBerat[i] < *bMin {
            *bMin = arrBerat[i]
        }

        if arrBerat[i] > *bMax {
            *bMax = arrBerat[i]
        }
    }
}

func rerata(arrBerat arrBalita) float64 {
    var n, i int
    var total float64

    n = int(arrBerat[0])

    for i = 1; i <= n; i++ {
        total += arrBerat[i]
    }

    return total / float64(n)
}

func main() {
    var balita arrBalita
    var n, i int
    var bMin, bMax, rata2 float64

    fmt.Print("Masukan banyak data berat balita : ")
    fmt.Scan(&n)

    balita[0] = float64(n)

    for i = 1; i <= n; i++ {
```

```

        fmt.Printf("Masukkan berat balita ke-%d: ", i)
        fmt.Scan(&balita[i])
    }

    hitungMinMax(balita, &bMin, &bMax)
    rata2 = rerata(balita)

    fmt.Printf("Berat balita minimum: %.2f kg\n", bMin)
    fmt.Printf("Berat balita maksimum: %.2f kg\n", bMax)
    fmt.Printf("Rerata berat balita: %.2f kg\n", rata2)
}

```

Output :

```

PS D:\ALPRO2\109082500026_Dwi-Sasmita-Agus-Anggraini\Week10-Modul10\Unguided\Unguided-3> go run Unguided3.go
Masukan banyak data berat balita : 4
Masukkan berat balita ke-1: 5.3
Masukkan berat balita ke-2: 6.2
Masukkan berat balita ke-3: 4.1
Masukkan berat balita ke-4: 9.9
Berat balita minimum: 4.10 kg
Berat balita maksimum: 9.90 kg
Rerata berat balita: 6.38 kg
PS D:\ALPRO2\109082500026_Dwi-Sasmita-Agus-Anggraini\Week10-Modul10\Unguided\Unguided-3>

```

Penjelasan :

Program ini menggunakan teknik unik di mana jumlah data n disimpan di indeks ke-0 array balita, sehingga array sekaligus membawa informasinya sendiri saat dikirim ke fungsi lain. Fungsi `hitungMinMax` menerima pointer `bMin` dan `bMax` agar nilainya bisa dimodifikasi langsung dari luar fungsi, sementara fungsi `rerata` cukup mengembalikan nilai rata-rata. Keduanya membaca jumlah data dari `arrBerat[0]` dan melakukan iterasi mulai indeks ke-1, lalu di main hasilnya ditampilkan dengan format dua angka desimal menggunakan.

D. Kesimpulan

Pencarian nilai ekstrim merupakan algoritma sederhana namun memiliki peran penting dalam kehidupan sehari-hari maupun dalam dunia pemrograman. Dengan cara memproses data secara sekuensial dan menyimpan nilai sementara untuk terus dibandingkan dengan data berikutnya, algoritma ini mampu menemukan nilai terkecil atau terbesar dari sekumpulan data secara efektif dan efisien.